

Object Oriented Programming (Classes-IV)

Abbas Khalid

April 21, 2025

Table of contents

- 1 Arrays as Data Members
- 2 Pointers as Data Members
- 3 The `this` Pointer

Example 1

```
1 class ArrayContainer {
2     static const int SIZE = 5;
3     int data[SIZE];
4 public:
5     ArrayContainer() {
6         for (int i = 0; i < SIZE; ++i)
7             data[i] = i + 1;
8     }
9     void display() const {
10         cout << "Array contents: \n";
11         for(int i = 0; i < SIZE; ++i)
12             cout << data[i] << " ";
13         cout << endl;
14     }
15 };
16
17 int main() {
18     ArrayContainer container;
19     container.display();
20     return 0;
21 }
```

Array contents:
1 2 3 4 5

Example 2

```

1  class ArrayContainer {
2      static const int SIZE = 5;
3      int data[SIZE];
4  public:
5      ArrayContainer() {
6          for (int i = 0; i < SIZE; ++i){
7              cout<<"Enter value ("<<i+1<<" / "<<SIZE<<") : ";
8              cin>>data[i];
9          }
10     }
11     void display() const {
12         cout <<"Array contents: \n";
13         for(int i = 0; i < SIZE; ++i) cout<<data[i]<<" ";
14         cout<<endl;
15     }
16 };
17 int main() {
18     ArrayContainer container;
19     container.display();
20     return 0;
21 }
    
```

```

Enter value (1/5):1
Enter value (2/5):2
Enter value (3/5):3
Enter value (4/5):4
Enter value (5/5):5
Array contents:
1 2 3 4 5
    
```

Example 3

```

1  class ArrayContainer {
2      static const int SIZE = 5;
3      int data[SIZE];
4  public:    //Following is complier-dependent
5      ArrayContainer():data{1,2,3,4,5} {}
6      void display() const {
7          cout <<"Array contents: \n";
8          for(int i = 0; i < SIZE; ++i)
9              cout<<data[i]<<" ";
10         cout<<endl;
11     }
12 };
13
14 int main() {
15
16     ArrayContainer container;
17     container.display();
18
19     return 0;
20 }

```

Array contents:
1 2 3 4 5

Example 4

```
1 class ArrayContainer {
2     static const int SIZE = 5;
3     int data[SIZE];
4 public:
5     ArrayContainer(int array[]) {
6         for(int i = 0; i < SIZE; ++i)
7             data[i] = array[i];
8     }
9     void display() const {
10         cout << "Array contents: \n";
11         for(int i = 0; i < SIZE; ++i)
12             cout << data[i] << " ";
13         cout << endl;
14     }
15 };
16 int main() {
17     int array[] = {1,2,3,4,5};
18     ArrayContainer container(array);
19     container.display();
20     return 0;
21 }
```

Array contents:
1 2 3 4 5

Example 5

```

1  class ArrayContainer {
2      static const int SIZE = 5;
3      int data[SIZE];
4  public:    //Following is complier-dependent
5      ArrayContainer(int a[]):data{a[0],a[1],a[2]}
6          {data[3] = a[3];data[4] = a[4];}
7      void display() const {
8          cout <<"Array contents: \n";
9          for(int i = 0; i < SIZE; ++i)
10             cout<<data[i]<<" ";
11             cout<<endl;
12     }
13 };
14
15 int main() {
16     int array[] = {1,2,3,4,5};
17     ArrayContainer container(array);
18     container.display();
19     return 0;
20 }

```

Array contents:
1 2 3 4 5

Example 1

```
1 class StringContainer {
2     static const int MAX_LENGTH = 100;
3     char data[MAX_LENGTH];
4 public:
5     StringContainer(const char* initData) {
6         for(int i = 0; i<MAX_LENGTH; ++i)
7             data[i] = '\0'; //null-termination
8         strcpy(data,initData);
9     }
10    void display() const
11        {cout<<"String contents: \n"<<data<<endl;}
12 };
13
14 int main() {
15
16     StringContainer container("Object Orientation");
17     container.display();
18
19     return 0;
20 }
```

String contents:
Object Orientation

Example 2

```
1 class StringContainer {
2     static const int MAX_LENGTH = 100;
3     char data[MAX_LENGTH];
4 public:
5     StringContainer(const char* initData) {
6         for(int i = 0; i<MAX_LENGTH; ++i)
7             data[i] = '\0'; //null-termination
8         strcpy(data,initData);
9     }
10
11     void display() const
12     {cout<<"String contents: \n"<<data<<endl;}
13 };
14
15 int main() {
16     const char initData[] = "Object Orientation";
17     StringContainer container(initData);
18     container.display();
19     return 0;
20 }
```

String contents:
Object Orientation

Example 3

```
1 class StringContainer {
2     static const int MAX_LENGTH = 50;
3     char data[MAX_LENGTH];
4 public:
5     StringContainer() {
6         cout<<"Enter a string: ";
7         cin.getline(data,50);
8     }
9
10    void display() const
11    {cout<<"String contents: "<<data<<endl;}
12 };
13
14 int main() {
15
16     StringContainer container;
17     container.display();
18
19     return 0;
20 }
```

Enter a String: Object Orientation
String contents: Object Orientation

Example 1

```

1  class PtrContainer{
2      int* ptr;
3  public:
4      PtrContainer(int value)
5          {ptr = &value;} //Problem?
6
7      int getValue()const{return *ptr;}
8      void setValue(int value){*ptr = value;}
9  };
10
11 int main() {
12
13     PtrContainer container(10);
14     cout<<"Initial value: "<<container.getValue()<<endl;
15
16     container.setValue(20);
17     cout<<"New value: "<<container.getValue()<<endl;
18
19     return 0;
20 }
```

Initial value: 10
 New value: 20

Example 2

```

1  class PtrContainer{
2      int* ptr;
3  public:
4      PtrContainer(int value)
5          {*ptr = value;} //Problem?
6
7      int getValue()const{return *ptr;}
8      void setValue(int value){*ptr = value;}
9  };
10
11 int main() {
12
13     PtrContainer container(10);
14     cout<<"Initial value: "<<container.getValue()<<endl;
15
16     container.setValue(20);
17     cout<<"New value: "<<container.getValue()<<endl;
18
19     return 0;
20 }

```

Initial value: 10
New value: 20

Pointers as Data Members: Potential Issues

Example 1: In the constructor `PtrContainer(int value)`, we are assigning the address of the local variable `value` to the `ptr` data member. This creates a problem because being a local variable within the constructor, memory of `value` will be deallocated when the constructor exits. Thus, `ptr` ends up pointing to a location that is no longer valid after the constructor completes. Accessing this pointer later on, such as in `getValue()`, may result in undefined behaviour, as it's pointing to a location that has been deallocated.

Example 2: When we attempt to assign a value to the memory location pointed to by `ptr`, we are assuming that `ptr` points to a valid memory location. However, in the constructor `PtrContainer(int value)`, `ptr` has not been initialized to point to any valid memory location. Therefore, dereferencing `ptr` and trying to assign a value to it will result in undefined behaviour. Valid sequence is as follows:
 (1) `int* ptr; int v;` (2) `ptr = &v;` (3) `*ptr = 10;`

Example 3

```

1  class PtrContainer{
2      int* ptr;
3  public:
4      PtrContainer(int *value)
5          {ptr = value;}    //Solution # 1
6
7      int getValue()const{return *ptr;}
8      void setValue(int value){*ptr = value;}
9  };
10
11 int main() {
12     int value = 10;
13     PtrContainer container(&value); //Note this.
14     cout<<"Initial value: "<<container.getValue()<<endl;
15
16     container.setValue(20);
17     cout<<"New value: "<<container.getValue()<<endl;
18
19     return 0;
20 }
```

Initial value: 10
 New value: 20

Example 4

```

1  class PtrContainer{
2      int* ptr;
3  public:
4      PtrContainer(int value)
5          {ptr = new int(value);} // Solution # 2
6
7      int getValue()const{return *ptr;}
8      void setValue(int value){*ptr = value;}
9      ~PtrContainer(){delete ptr;}
10 };
11
12 int main() {
13     PtrContainer container(10);
14     cout<<"Initial value: "<<container.getValue()<<endl;
15
16     container.setValue(20);
17     cout<<"New value: "<<container.getValue()<<endl;
18
19     return 0;
20 }

```

Initial value: 10
New value: 20

Example 5

```

1  class PtrContainer{
2      int* ptr;
3  public:
4      PtrContainer(int *value){ptr = value;}
5      int* getValue()const{return ptr;}
6      void setValue(int value){*ptr = value;}
7  };
8
9  int main() {
10     int value = 10;
11     PtrContainer container(&value);
12     int* p = container.getValue();
13     cout<<"Initial value: "<<*p<<endl;
14
15     container.setValue(20);
16     p = container.getValue();
17     cout<<"New value: "<<*p<<endl;
18
19     return 0;
20 }
```

Initial value: 10
 New value: 20

Example 6

```

1  class PtrContainer{
2      int* ptr;
3  public:
4      PtrContainer(int *value){ptr = value;}
5      int* getValue()const{return ptr;}
6  };
7
8  int main() {
9
10     int value = 10;
11     PtrContainer container(&value);
12     int* p = container.getValue();
13     cout<<"Initial value: "<<*p<<endl;
14     *p = 20; //Encapsulation violates
15     p = container.getValue();
16     cout<<"New value: "<<*p<<endl;
17
18     return 0;
19 }
```

Initial value: 10
 New value: 20

Example 7

```

1  class PtrContainer{
2      int* ptr;
3  public:
4      PtrContainer(int *value){ptr = value;}
5      int* getValue()const{    // Solution
6          int* temp = new int; *temp = *ptr;
7          return temp;
8      }
9 };
10
11 int main() {
12     int value = 10;
13     PtrContainer container(&value);
14     int* p = container.getValue();
15     cout<<"Initial value: "<<*p<<endl;
16     *p = 20; //Encapsulation is intact
17     p = container.getValue();
18     cout<<"New value: "<<*p<<endl;
19
20     return 0;
21 }
    
```

Initial value: 10
 New value: 10

Example 8

```

1  class PtrContainer{
2      int* ptr;
3  public:
4      PtrContainer(int *value){ptr = value;}
5      int* getValue()const{
6          int* temp = new int; *temp = *ptr;
7      return temp;
8      }
9      void setValue(int value){*ptr = value;}
10 };
11
12 int main() {
13     int value = 10;
14     PtrContainer container(&value);
15     int* p = container.getValue();
16     cout<<"Initial value: "<<*p<<endl;
17     container.setValue(20);
18     p = container.getValue();
19     cout<<"New value: "<<*p<<endl;
20     return 0;
21 }
    
```

Initial value: 10
 New value: 20

Example 1

```

1 class SPtrContainer {
2     static int* staticPtr;
3
4     public:
5         SPtrContainer(){}
6         ~SPtrContainer(){delete staticPtr;} //Problem?
7         static int getValue(){return *staticPtr;}
8         static void setValue(int value){*staticPtr = value;}
9 };
10 int* SPtrContainer::staticPtr = new int(10);
11 int main() {
12     SPtrContainer container1, container2;
13     cout<<"Container1:"<<container1.getValue()<<endl;
14     cout<<"Container2:" <<container2.getValue()<<endl;
15
16     SPtrContainer::setValue(20);
17
18     cout<<"Container1:"<<container1.getValue()<<endl;
19     cout<<"Container2:" <<container2.getValue()<<endl;
20     return 0;
21 }

```

Container1: 10
Container2: 10
Container1: 20
Container2: 20

Example 2

```

1 class SPtrContainer {
2     static int* staticPtr;
3
4 public:
5     SPtrContainer(){}
6     ~SPtrContainer(){} //Solution - static are not at object level
7     static int getValue(){return *staticPtr;}
8     static void setValue(int value){*staticPtr = value;}
9 };
10 int* SPtrContainer::staticPtr = new int(10);
11 int main() {
12     SPtrContainer container1, container2;
13     cout<<"Container1:"<<container1.getValue()<<endl;
14     cout<<"Container2:" <<container2.getValue()<<endl;
15
16     SPtrContainer::setValue(20);
17
18     cout<<"Container1:"<<container1.getValue()<<endl;
19     cout<<"Container2:" <<container2.getValue()<<endl;
20     return 0;
21 }

```

```

Container1: 10
Container2: 10
Container1: 20
Container2: 20

```

Example 3

```

1  class SPtrContainer {
2      static int* staticPtr;
3
4  public:
5      SPtrContainer(){}
6      SPtrContainer(int a){staticPtr = new int(a);}
7      ~SPtrContainer(){} //Whenever an object destroys, it executes
8      static int getValue(){return *staticPtr;}
9      static void setValue(int value){*staticPtr = value;}
10 };
11 int* SPtrContainer::staticPtr;
12 int main() {
13     SPtrContainer container1(10), container2;
14     cout<<"Container1:"<<container1.getValue()<<endl;
15     cout<<"Container2:" <<container2.getValue()<<endl;
16     SPtrContainer::setValue(20);
17     cout<<"Container1:"<<container1.getValue()<<endl;
18     cout<<"Container2:" <<container2.getValue()<<endl;
19     return 0;
20 }

```

```

Container1: 10
Container2: 10
Container1: 20
Container2: 20

```

Example 1

```

1 class CPtrContainer {
2     int* const constPtr;
3 public:
4     CPtrContainer(int* val):constPtr(val) {}
5     int getValue() const {return *constPtr;}
6     void setValue(int val) {*constPtr = val;}
7 };
8
9 int main() {
10
11     int value = 10;
12     CPtrContainer container(&value);
13     cout<<"Initial value: "<<container.getValue()<<endl;
14
15     //Trying to modify a Constant Pointer
16     //container.setValue(20); Error!
17
18     return 0;
19 }

```

Initial value: 10

Example 2

```

1  class CPtrContainer {
2      int value;
3      int* const constPtr;
4  public:
5      CPtrContainer(int val):
6          value(val), constPtr(new int(val)) {}
7      ~CPtrContainer() {delete constPtr;}
8      int getValue() const {return *constPtr;}
9      void setValue(int val) {*constPtr = val;}
10 };
11
12 int main() {
13
14     CPtrContainer container(10);
15     cout<<"Initial value: "<<container.getValue()<<endl;
16
17     //Trying to modify a Constant Pointer
18     //container.setValue(20); Error!
19
20     return 0;
21 }

```

Initial value: 10

The `this` Pointer

- Every object in C++ has access to its own address through an important pointer called `this` pointer.
- The `this` pointer is an implicit parameter to all `member functions`. Therefore, inside a member function, `this` may be used to refer to the invoking object.
- When the local variable's name is same as a class member's name. The class member's name become invisible in that scope. The `this` pointer is used to access the hidden name.
- Friend functions do not have a `this` pointer, because friends are not members of a class. Only member functions have a `this` pointer.
- The `this` pointer refers to the current object instance within non-static member functions. It holds the memory address of the object on which the member function is being called. The `static` member functions operate at the class level rather than the object level. So, the `this` pointer is not available within `static` member functions.

Example 1: Accessing the Hidden Class Member

```
1  /* When local variable's name is same as a class member's name. The class
   member's name become invisible in that scope. The 'this' pointer is used
   to access the hidden name.*/
2
3  class Test {
4      int x;
5  public:
6      void setX (int x){this->x = x;}
7      void print() {cout <<" x = " << x << endl;}
8  };
9
10 int main(){
11
12     Test obj;
13     obj.setX(20); obj.print();
14
15     return 0;
16 }
```

x: 20

Example 2: Referring to the Invoking Object

```

1  class A{
2      int var;
3  public:
4      A(int value){var = value;}
5      A* get(){return this;}
6      int getVar(){return var;}
7  };
8
9  int main(){
10     A obj(10),*objPtr;
11     objPtr = obj.get();
12     cout<<"Address of obj: "<<&obj<<endl;
13     cout<<"Value of var: "<<obj.getVar()<<endl;
14     cout<<"Address stored in objPtr: "<<objPtr<<endl;
15     cout<<"Value of var: "<<objPtr->getVar()<<endl;
16
17     return 0;
18 }

```

```

Address of obj: 0x6ffe00
Value of var: 10
Address stored in objPtr: 0x6ffe00
Value of var: 10

```

Example 3: Referring to the Invoking Object

```

1  class A{
2      int var;
3      public:
4
5      //The 'this' pointer belongs to the object through which a function is called.
6
7      A(int value){this->var = value;}
8      void setVar(int value){this->var = value;}
9      int getVar(){return this->var;}
10 };
11
12 int main(){
13
14     A obj(5);
15     cout<<"Value of var: "<<obj.getVar()<<endl;
16     obj.setVar(10);
17     cout<<"Value of var: "<<obj.getVar()<<endl;
18
19     return 0;
20 }

```

Value of var: 5
Value of var: 10

Example 4: Referring to the Invoking Object

```
1 class Box {
2     double length, breadth, height;
3 public:
4     Box(double l = 2.0, double b = 2.0, double h = 2.0)
5         {length = l; breadth = b; height = h;}
6     double Volume() {return length * breadth * height;}
7     int compare(Box b){return this->Volume()>b.Volume();}
8 };
9
10 int main() {
11
12     Box Box1(3.3, 1.2, 1.5);
13     Box Box2(8.5, 6.0, 2.0);
14
15     if(Box1.compare(Box2))
16         cout<<"Box2 is smaller than Box1"<<endl;
17     else
18         cout<<"Box2 is equal to or larger than Box1"<<endl;
19
20     return 0;
21 }
```

Box2 is equal to or larger than Box1

Example 5: The this Pointer and static Data Members

```
1 class A{
2     static int var;
3     public:
4
5     A(int value){this->var = value;}
6     void setVar(int value){this->var = value;}
7     int getVar(){return this->var;}
8 };
9
10 int A::var;
11
12 int main(){
13
14     A obj(5);
15     cout<<"Value of var: "<<obj.getVar()<<endl;
16     obj.setVar(10);
17     cout<<"Value of var: "<<obj.getVar()<<endl;
18
19     return 0;
20 }
```

Value of var: 5
Value of var: 10

Example 6: The this Pointer and static-Member Functions

```

1  class A{
2      static int var;
3  public:
4      //The 'this' pointer is unavailable for static-member functions
5      A(int value){this->var = value;}
6      static void setVar(int value){this->var = value;}// Error
7      static int getVar(){return this->var;}// Error
8  };
9  int A::var;
10 int main(){
11
12     A obj(5);
13     cout<<"Value of var: "<<obj.getVar()<<endl;
14     obj.setVar(10);
15     cout<<"Value of var: "<<obj.getVar()<<endl;
16
17     return 0;
18 }
    
```

Example 7: The this Pointer and non-Member Functions

```
1 class A{
2     int var;
3 };
4
5 //The 'this' pointer is unavailable for non-member functions
6 A* getThis( ){return this;} // Error
7
8 int main(){
9
10    cout<<"Value of this: "<<getThis()<<endl;
11
12    return 0;
13 }
```


Conclusions

In this lecture we discussed about

- Arrays as Data Members
- Pointers as Data Members
- The `this` pointer

In the next lecture we will discuss about

- Array of Objects
- Pointers to Objects
- Pointers to Array of Objects
- Shallow Copy & Deep Copy